UPSA
UNIVERSITY OF PROFESSIONAL STUDIES, ACCRA
Scholarship with Professionalism

NETWORK AND SYSTEMS ADMINISTRATION

WEEK 7: EMERGING INFRASTRUCTURE SYSTEMS

Cloud Computing • Virtualization • Containers • Virtual Networks • Storage • Shared Responsibility

Practical Environment: Kali Linux VM

Dr Samuel Attuquayefio | Mr. Adams Azamati

WEEK 7 ROADMAP

From one physical host to flexible, shared infrastructure

1	2	3	4	5
Physical Host	**Hypervisor**	**Virtual Machine**	**Container**	**Cloud Platform**
Hardware is directly administered	Hardware resources are abstracted	Guest OS gets virtual hardware	Application is isolated while sharing a kernel	Resources are consumed as services

INDIVIDUAL PRACTICAL

Identify the virtualisation environment. Where available, inspect a container runtime, running containers and local images.

GROUP ACTIVITY

Compare one organisational service on a physical server, VM, container and cloud platform, then justify the best option.

LEARNING OUTCOMES

By the end of Week 7, students should be able to:

- Explain virtual machines, hypervisors and containers from an administrator's perspective.
- Distinguish physical-server, VM, container and cloud-platform deployment approaches.
- Explain major cloud service models and how administrative control changes across them.
- Explain the administrative implications of virtual networks and virtual storage.
- Explain shared responsibility and identify responsibilities retained by the customer/administrator.
- Use prescribed Linux commands to identify virtualisation and inspect Docker or Podman where available.
- Compare deployment options for one organisational service using appropriate administrative criteria.
- Where facilities permit, deploy or inspect a small containerised service and document the evidence.

WHY INFRASTRUCTURE BECAME VIRTUAL

Traditional one-service-per-server designs often left expensive resources underused.

PHYSICAL LIMIT

One server has fixed CPU, memory, storage and network hardware. Capacity changes may require procurement and installation.

VIRTUALISATION IDEA

Abstract physical resources so several isolated computing environments can share the same host.

ADMIN BENEFIT

Provision faster, consolidate workloads, isolate services and manage resources more flexibly.

Abstraction changes the administrator's view: you manage both the physical capacity and the virtual resources built on top of it.

VIRTUALISATION: THE CORE IDEA

Virtualisation separates the logical computing environment from the underlying physical hardware.

- Physical resources include CPU, memory, storage and network interfaces.
- A virtualisation layer allocates or presents these resources to one or more guest environments.
- Each guest can be administered as a distinct system even though resources are shared underneath.
- Isolation reduces direct interference between workloads, but the host remains a common dependency.
- Administrators must manage capacity, availability, security and lifecycle at more than one layer.

ADMIN QUESTION

What is physical, what is virtual, and which layer owns the problem?

KEY RISK

A host failure can affect several virtual machines at once.

HYPERVISOR: THE VIRTUALISATION CONTROL LAYER

The hypervisor creates and manages virtual machines and mediates access to host resources.

The hypervisor allocates virtual CPU, memory, disk and network resources to each VM.

VIRTUAL MACHINE	VIRTUAL MACHINE	VIRTUAL MACHINE
VM 1 Guest OS + App	VM 2 Guest OS + App	VM 3 Guest OS + App

HYPERVISOR

PHYSICAL HARDWARE: CPU • MEMORY • STORAGE • NETWORK

TYPE 1 AND TYPE 2 HYPERVISORS

The placement of the hypervisor changes the architecture and typical use case.

Feature	Type 1 / Bare-metal	Type 2 / Hosted
Runs on	Directly on server hardware	On top of a host operating system
Typical use	Data centres and production virtualisation	Desktop labs, development and learning
Management focus	Host clusters, capacity, availability	Host OS plus desktop virtualisation
Examples	KVM-based platforms, VMware ESXi, Hyper-V Server-style deployments	VirtualBox, VMware Workstation/Fusion
Common dependency	Hypervisor host and shared infrastructure	Host OS plus hypervisor application

VIRTUAL MACHINE

A VM behaves like a computer because it receives virtual hardware and runs its own guest operating system.

- Has its own guest operating system, kernel, processes, users and services.
- Receives virtual CPU, memory, disk and network interfaces from the virtualisation platform.
- Can often run a different operating system from other VMs on the same host.
- Can be started, stopped, cloned, snapshotted or migrated depending on platform capabilities.
- Requires operating-system administration inside the guest in addition to host/platform administration.

ADMIN RESPONSIBILITY

Patch the guest OS, manage users/services, monitor resources and protect data.

DEPENDENCY

Guest availability still depends on host, storage and virtual networking.

CONTAINERS: OS-LEVEL ISOLATION

Containers isolate application environments while sharing the host operating-system kernel.

No separate guest kernel is required for each container.

ISOLATED ENVIRONMENT

Container A
App + dependencies

ISOLATED ENVIRONMENT

Container B
App + dependencies

ISOLATED ENVIRONMENT

Container C
App + dependencies

CONTAINER RUNTIME

HOST OPERATING SYSTEM + SHARED KERNEL

PHYSICAL / VIRTUAL HOST

VIRTUAL MACHINES VS CONTAINERS

Both isolate workloads, but they isolate at different layers.

Criterion	Virtual Machine	Container
Kernel	Each VM has its own guest kernel	Containers share the host kernel
Image size	Usually larger OS image	Usually smaller application image
Start-up	Typically slower than a container	Typically fast
Isolation boundary	Virtual hardware + guest OS	Processes, namespaces/cgroups and runtime controls
OS flexibility	Different guest OSs may coexist	Must be compatible with host kernel family
Administration	Guest OS + application + platform	Host/runtime + image + application

CONTAINER IMAGE, CONTAINER AND RUNTIME

Three terms students must not confuse.

IMAGE	CONTAINER	RUNTIME / ENGINE
A packaged, read-only template containing application files, dependencies and metadata used to create containers.	A running or stopped instance created from an image, with its own isolated process environment.	Software such as Docker or Podman that manages images and container lifecycle operations.

Image → create/run → container. One image can be used to create multiple containers.

CLOUD COMPUTING

Cloud computing provides computing resources as services that can be provisioned without owning or directly handling all underlying hardware.

- Compute, storage, networking and applications can be consumed as services.
- Resources can often be provisioned and adjusted on demand through portals, APIs or automation.
- The customer may never see the physical server that delivers the service.
- Cloud does not eliminate administration; it redistributes administrative responsibility.
- Cost, availability, scalability, security, data location and dependency on providers must be evaluated.

IMPORTANT

Virtualisation is a technology; cloud computing is a service-delivery and operating model that may use

ADMIN VIEW

You manage through abstractions, policies, identities, virtual networks, storage and provider services.

CLOUD SERVICE MODELS

The service model determines how much of the technology stack the customer administers.

Layer	IaaS	PaaS	SaaS
Application	Customer	Customer	Provider
Data / configuration	Customer	Customer	Customer configuration/data
Runtime / middleware	Customer	Provider	Provider
Operating system	Customer	Provider	Provider
Virtualisation / hardware	Provider	Provider	Provider
Administrator focus	VMs, OS, virtual network, storage	Application, data, service configuration	Users, data, configuration, access and governance

INFRASTRUCTURE AS A SERVICE (IaaS)

The customer administers virtual infrastructure while the provider operates the underlying physical platform.

- Provision virtual machines, virtual networks and storage without purchasing each physical server.
- Customer typically manages the guest OS, patches, users, services, firewall/security configuration and data.
- Provider manages data-centre facilities, physical hardware and core virtualisation platform.
- Useful when the organisation needs substantial operating-system and network control.
- Administrative skill remains close to traditional systems administration, but through cloud interfaces.

CHECK YOUR UNDERSTANDING

Who patches the guest operating system? Usually the customer.

ADMIN EVIDENCE

Configuration, access records, policies, monitoring and provider documentation.

PLATFORM AS A SERVICE (PaaS)

The provider manages more of the operating stack so the customer can focus on applications and data.

- Customer deploys code or applications to a managed platform.
- Provider commonly manages the OS, runtime, scaling mechanisms and underlying infrastructure.
- Administrators focus more on application configuration, identities, data, connectivity and service policies.
- Less low-level control can simplify operations but may constrain customisation.
- Provider-specific services can create operational dependency or migration challenges.

CHECK YOUR UNDERSTANDING

Which responsibilities disappear, and which simply move to a different layer?

ADMIN EVIDENCE

Configuration, access records, policies, monitoring and provider documentation.

SOFTWARE AS A SERVICE (SaaS)

The organisation consumes a complete application operated by the provider.

- Users access an application without administering its servers or operating system.
- Customer still manages user lifecycle, access rights, data handling, configuration and organisational policy.
- Availability and platform maintenance depend heavily on the provider.
- Integration, backup/export options, audit logs and data governance remain administrative concerns.
- SaaS can reduce infrastructure work but does not remove accountability for organisational use.

CHECK YOUR UNDERSTANDING

If the provider runs the application, who manages your staff accounts and permissions?

ADMIN EVIDENCE

Configuration, access records, policies, monitoring and provider documentation.

SHARED RESPONSIBILITY

Moving to cloud changes responsibility; it does not remove it.

PROVIDER TYPICALLY SECURES

- Physical data centre
- Physical servers and core networking
- Hypervisor/platform components within the service boundary
- Availability of contracted provider services
- Provider-side controls defined by the service

CUSTOMER STILL SECURES

- Identities and access
- Data and classification
- Guest OS in IaaS
- Application/configuration choices
- Virtual network/security rules
- Credentials, keys and secrets
- Compliance with organisational requirements

The exact boundary depends on the service model and provider agreement. Never assume "the cloud provider handles security."

VIRTUAL NETWORKS

Virtual machines and cloud resources still need addressing, routing, segmentation and controlled connectivity.

- Virtual NICs connect VMs or containers to virtual switches, bridges or software-defined networks.
- Virtual networks can create isolated segments without requiring a separate physical switch for each segment.
- Administrators still manage IP addresses, routes, DNS, firewall/security rules and connectivity dependencies.
- NAT, bridged networking and provider virtual networks can make the same VM appear differently from the guest perspective.
- Network documentation must show both logical/virtual connections and critical physical/provider dependencies.

LINK TO WEEK 4

The networking concepts remain relevant; the implementation is now partly virtual.

ADMIN QUESTION

Which virtual network path must succeed for the service to be reachable?

VIRTUAL STORAGE

A virtual disk looks like a disk to the guest, but its data may live on files, volumes, SANs or cloud storage underneath.

- VM disks are commonly presented as virtual block devices to the guest operating system.
- The underlying storage may be local to the host or shared across multiple hosts.
- Containers usually need explicit persistent storage when application data must survive container replacement.
- Cloud platforms may provide block, file and object storage with different administrative properties.
- Capacity, performance, backup, encryption, persistence and recovery must be planned at the correct layer.

CRITICAL DISTINCTION

A VM snapshot is not automatically a complete backup strategy.

ADMIN QUESTION

Where does the data persist if the VM or container is deleted?

THE VIRTUAL INFRASTRUCTURE STACK

Compute, network and storage are all abstracted, but all still depend on real resources.

WORKLOADS

Applications • services • containers

VIRTUAL COMPUTE

VMs • vCPU • vMemory

CLOUD SERVICES

IaaS • PaaS • SaaS

VIRTUAL NETWORK

vNIC • vSwitch • routes • security rules

VIRTUAL STORAGE

Virtual disks • volumes • persistent data

CONTROL PLANE

Portal • API • identity • monitoring

PHYSICAL / PROVIDER INFRASTRUCTURE

BENEFITS OF VIRTUALISATION AND CLOUD

The value comes from flexibility, consolidation and faster service delivery, not from technology for its own sake.

- Consolidation: several workloads can share a physical host more efficiently.
- Rapid provisioning: VMs or cloud resources can often be created faster than new physical servers can be procured.
- Isolation: workloads can be separated into distinct environments.
- Portability and repeatability: images/templates support consistent deployment.
- Scalability: cloud capacity can often be adjusted as demand changes.
- Recovery options: snapshots, images, replication and migration may support resilience when properly designed.

CAUTION

Every benefit introduces new management layers, dependencies and skills.

DECISION RULE

Choose technology because it fits the service requirement, not because it is newer.

RISKS AND ADMINISTRATIVE TRADE-OFFS

Abstraction can hide complexity as well as simplify it.

- Host concentration: one physical failure may affect many virtual workloads.
- Resource contention: CPU, memory, storage I/O or network capacity may be oversubscribed.
- VM/container sprawl: easy provisioning can create forgotten resources and security exposure.
- Configuration complexity: failures can cross guest, hypervisor, virtual network, storage and cloud layers.
- Provider dependency: outages, service changes, pricing and platform-specific features affect operations.
- Security boundaries: misconfiguration, weak identities or exposed services remain customer risks.

ADMIN MINDSET

Inventory • monitor • control change • document dependencies • test recovery

KEY QUESTION

What new single point of failure did this design create?

INDIVIDUAL PRACTICAL: IDENTIFY THE ENVIRONMENT

The Week 7 practical begins with evidence, not assumptions.

- Run the prescribed commands in the Linux VM.
- Record the exact command and relevant output.
- Identify what the output proves and what it does not prove.
- Where Docker is unavailable, use Podman if it is the available runtime.
- Where neither runtime is installed, document that fact rather than inventing output.
- Where facilities permit, inspect or run a small containerised service.

PRESCRIBED

systemd-detect-virt
lscpu | grep -i hypervisor

RUNTIME CHECK

docker … OR podman …
depending on availability

COMMAND 1: systemd-detect-virt

Ask the operating system whether systemd detects a supported virtualisation environment.

$ systemd-detect-virt

- Returns a virtualisation/container identifier when a recognised environment is detected.
- The exact result depends on the hypervisor or container environment.
- A result of none means systemd did not identify supported virtualisation in that context.
- Record the returned value exactly; do not replace it with the platform you expected to see.
- Use the result together with other evidence rather than as the only source of truth.

EVIDENCE QUESTION

What virtualisation technology does the guest detect?

INTERPRETATION

Detection identifies environment type; it does not describe all host configuration.

COMMAND 2: lscpu | grep -i hypervisor

Look for hypervisor-related CPU information exposed to the guest.

$ lscpu | grep -i hypervisor

- lscpu reports CPU architecture information available to the Linux system.
- grep -i hypervisor filters lines containing the word “hypervisor” without case sensitivity.
- Output may identify a hypervisor vendor or virtualisation-related field when exposed to the guest.
- No matching line is still an observation and should be recorded honestly.
- Compare this evidence with systemd-detect-virt rather than assuming the outputs must be identical.

PIPE REVIEW

The | sends lscpu output into grep for filtering.

ADMIN HABIT

Filter large output, but keep enough context to interpret it correctly.

COMMAND 3: docker --version OR podman --version

Determine whether a supported container command-line runtime is available.

$ docker --version OR $ podman --version

- Use the command for the runtime installed in the lab environment.
- A version response confirms that the client command is installed and identifies its version.
- A “command not found” message means that command is not available through the current shell path.
- Do not install software during the activity unless the lecturer has authorised installation.
- Record the runtime and version because administration procedures may vary by implementation/version.

OUTLINE RULE

Docker OR Podman. Use what is actually available.

CAUTION

A client version alone does not prove that a container daemon/service is usable.

COMMAND 4: docker ps OR podman ps

Inspect containers currently known/running under the selected runtime.

$ docker ps OR $ podman ps

- Lists running containers by default.
- Important fields may include container ID, image, command, creation/status, ports and name.
- An empty list can be a valid result: the runtime may be working with no running containers.
- Permission or daemon/socket errors must be recorded and interpreted rather than hidden.
- Container status is operational evidence, similar to checking services and processes in earlier weeks.

ASK

Is a container running? Which image created it?

LINK TO WEEK 3

A container is also part of service/process administration.

COMMAND 5: docker images OR podman images

Inspect container images stored locally.

$ docker images OR $ podman images

- Lists images available locally to the selected runtime.
- Fields commonly include repository/name, tag, image ID, creation information and size.
- Images are templates; they are not the same as running containers.
- Multiple tags may refer to related image content, so interpret IDs and tags carefully.
- Unused images consume storage and should be managed deliberately in real environments.

COMPARE

docker ps → containers
docker images → images

EVIDENCE

Record image name/tag and size where present.

WHERE FACILITIES PERMIT: SMALL CONTAINERISED SERVICE

The outline permits a small deployment where the laboratory supports it.

- Use only a lecturer-approved image already available locally or obtainable through the authorised lab network.
- Start the container with a clear name and only the ports/resources required for the demonstration.
- Verify that the container appears in the runtime's process/container listing.
- Test the service from the appropriate host/client location.
- Stop and remove the test container when the exercise is complete, unless it is needed for later evidence.
- Record the command, output, test result and any corrective action.

LECTURER CONTROL

Do not require Internet downloads if the lab cannot support them.

SUCCESS CRITERION

Running container + reachable service + documented evidence.

WEEK 7 PRACTICAL EVIDENCE

Convert terminal output into an administrator's record.

Evidence	Command	Interpretation
Virtualisation type	systemd-detect-virt	Detected environment or none
Hypervisor CPU evidence	lscpu | grep -i hypervisor	Hypervisor-related field, or no match
Runtime/version	docker --version OR podman --version	Available runtime and version
Running containers	docker ps OR podman ps	Container state, image and ports where present
Local images	docker images OR podman images	Image names/tags/IDs/sizes where present
Optional service	Lecturer-approved deployment	Container + service verification evidence

GROUP ACTIVITY: ONE SERVICE, FOUR DEPLOYMENT OPTIONS

Compare the same organisational service fairly before choosing a platform.

- Use the small Ghanaian organisation selected by the group earlier in the course, or another lecturer-approved scenario.
- Select one organisational service, for example a web application, records system, file service or internal business application.
- Compare deployment on: physical server, virtual machine, container and cloud platform.
- Consider administration, cost/resource use, isolation, scalability, availability, security, portability and required skills.
- Justify the most appropriate option for the organisation rather than choosing the newest technology.
- Where facilities permit, deploy a small containerised service as practical evidence.

6–7 GROUPS

Keep the organisational scenario small enough for a clear comparison.

OUTPUT

Comparison matrix + recommendation + short justification.

GROUP COMPARISON MATRIX

Use the same criteria for all four options.

Criterion	Physical Server	VM	Container	Cloud Platform
Provisioning speed				
Resource efficiency				
OS control / isolation				
Scaling flexibility				
Network/storage administration				
Security responsibility				
Backup/recovery				
Cost/skills/complexity				

Groups should write short evidence-based judgments, not simply “good” or “bad.”

GHANAIAN CASE: SMALL CLINIC APPOINTMENT SERVICE

Which deployment option best fits a growing clinic with limited in-house IT staff?

PHYSICAL SERVER	VIRTUAL MACHINE	CONTAINER	CLOUD PLATFORM
High direct control. Requires hardware purchase, local power/cooling, maintenance and recovery planning.	Flexible OS environment. Still requires guest patching, virtual network/storage and host/platform administration.	Efficient application packaging. Requires runtime/image skills and deliberate persistent storage/network design.	Fast provisioning and remote reach. Requires Internet/provider dependency, cost control, identity/security and data governance.

Recommendation must follow the clinic's actual requirements: availability, data sensitivity, connectivity, budget, skills, growth and recovery needs.

GROUP CHECK: WHO IS RESPONSIBLE?

For the chosen option, assign every important task to an owner.

- 1. Physical hardware failure
- 2. Guest OS patching
- 3. Container image updates
- 4. User identities and permissions
- 5. Virtual network/security rules
- 6. Application configuration
- 7. Data backup and restoration
- 8. Monitoring and incident response

LABEL EACH TASK

Organisation / administrator
Provider / platform
Shared / depends on service model

WHY THIS MATTERS

Unassigned responsibility becomes an operational and security gap.

GROUP PRESENTATION: 4 MINUTES EACH

Present the decision as an infrastructure administrator.

- 30 sec — Organisation and selected service.
- 60 sec — Compare physical server, VM, container and cloud platform.
- 45 sec — Explain virtual network and storage implications.
- 45 sec — Explain shared-responsibility implications.
- 30 sec — State the recommended option and two reasons.
- 30 sec — Identify one major risk and its administrative control.

LECTURER CHECK

Did the group compare all four options rather than describe only its favourite?

PEER QUESTION

What assumption, cost or responsibility could change the recommendation?

DEPLOYMENT DECISION GUIDE

Choose the option that best matches the service requirement and administrative capability.

PHYSICAL	VM	CONTAINER	CLOUD
Consider when direct hardware control, specialised hardware or strict local requirements dominate.	Consider when strong OS separation, legacy compatibility or full guest control is important.	Consider when lightweight, repeatable application deployment and fast lifecycle are valuable.	Consider when on-demand capacity, managed services, geographic reach or rapid scaling are valuable.

The “best” platform is conditional. Administrators must balance service needs, security, skills, resilience, cost and dependency.

WEEK 7 COMMAND RECAP

Each command answers a specific administrative question.

Command	Administrative question
systemd-detect-virt	What virtualisation environment does systemd detect?
lscpu | grep -i hypervisor	What hypervisor-related CPU evidence is exposed?
docker --version / podman --version	Which container runtime command/version is available?
docker ps / podman ps	Which containers are running?
docker images / podman images	Which images are stored locally?

WEEK 7 KEY TAKEAWAYS

Emerging infrastructure changes the layers we administer, not the need for disciplined administration.

- Virtual machines use virtual hardware and normally run their own guest operating-system kernel.
- Hypervisors allocate and control virtual machine access to physical resources.
- Containers isolate application environments while sharing a host kernel.
- Cloud service models redistribute control and responsibility across customer and provider.
- Virtual networks and storage still require addressing, connectivity, capacity, security, persistence and documentation.
- Shared responsibility must be explicit; unassigned tasks become operational gaps.
- Deployment decisions should be justified from organisational requirements, not technology fashion.

WEEK 7 EVIDENCE & READING

Retain both technical evidence and decision-making evidence.

INDIVIDUAL EVIDENCE

- Virtualisation detection output
- Hypervisor-related CPU evidence
- Docker/Podman version where available
- Container listing where available
- Image listing where available
- Optional containerised-service evidence
- Short interpretation for every result

GROUP EVIDENCE

- Selected organisational service
- Four-option comparison matrix
- Virtual network/storage implications
- Shared-responsibility allocation
- Recommended deployment option
- Justification and major risk/control

Prescribed reading: Nemeth et al., 5th ed., Ch. 24 Virtualization, pp. 900–914; Ch. 25 Containers, pp. 915–948. Ward (2021), Ch. 15 Virtualization, pp. 401–418.